03 Kombinatorik

03_combinational, p.1

Variablenbelegung

- Eine Zeile der Tafel
- Bsp: 3 Variablen $x_1, x_2, x_3 \rightarrow 2^3 = 8$ Variablenbelegungen
- Eine Variablenbelegung ist z.B.: $x_1 = 0; x_2 = 0; x_3 = 1$

Grundlegender Aufbau: Verhalten → Wahrheitstafel → Normalform (DNF/KNF) → Vereinfachen → Schaltung

Das Verhalten wird in einer Wahrheitstabelle dargestellt.
Aus dieser wollen wir jetzt die Normalform bestimmen:

Normalformen

03_combinational, p.7
- Zum bestimmen der Normalform brauchen wir die Min und Maxterme der booleschen Funktion

Minterm (03_combinational, p.9)

- Verknüpfung aller Variablen mit UND
- Jede Variable kommt einmal vor
- Entweder normal x oder negiert $\overline{x}$
- **Eine** bestimmte Kombination wo das Ergebnis 1 ist → $y = 1$
  Beispiel:
- $x_1 = 0; x_2 = 0; x_3 = 1 \rightarrow 001$
- Mindterm: $\overline{x_2} \wedge \overline{x_1} \wedge x_0$
- da $\overline{0} \wedge \overline{0} \wedge 1 = 1 \wedge 1 \wedge 1 = 1$
- Jede andere Belegung ist mindestens eine Belegung 0

x_2	x_1	x_0	y	Minterm
0	0	0	0	(leer)
0	0	1	1	¬x2 ¬x1 x0
0	1	0	0	(leer)
0	1	1	0	(leer)
1	0	0	1	x2 ¬x1 ¬x0
1	0	1	1	x2 ¬x1 x0
1	1	0	1	x2 x1 ¬x0
1	1	1	0	(leer)

Maxterm

- Gegenteil Minterm
- verknüpft alle Variablen eines Terms mit ODER ($\vee$)
- **Eine** bestimmte Kombination/Variablenbelegung wo das Ergebnis 0 ist
- Beispiel: 001 $\rightarrow$ y = 0 $\rightarrow$ Maxterm: $x_2 \vee \overline{x_1} \vee \overline{x_0}$

	Minterm	**Maxterm**
Verknüpfung	UND ( )	ODER ( )
Wird 1 bei…	**genau einer** Belegung	**genau einer** Belegung
Wird 0 bei…	allen anderen	allen anderen
Regel fürs Literal	`x` wenn Eintrag **1**, `¬x` wenn **0**	`x` wenn Eintrag **0**, `¬x` wenn **1**

Disjunktive Normalform

03_combinational, p.12
Errinnerung: Disjunktion = ODER
- Alle Minterme (y=1) mit ODER verknüpfen
- $y = (\overline{x_2}\,\overline{x_1}x_0) \vee (x_2\overline{x_1}\,\overline{x_0}) \vee (x_2\overline{x_1}x_0) \vee (x_2x_1\overline{x_0})$
- Das stimmt immer, weil y genau dann 1 ist, wenn einer der Detektoren anschlägt.
- DNF = ODER von UNDs.

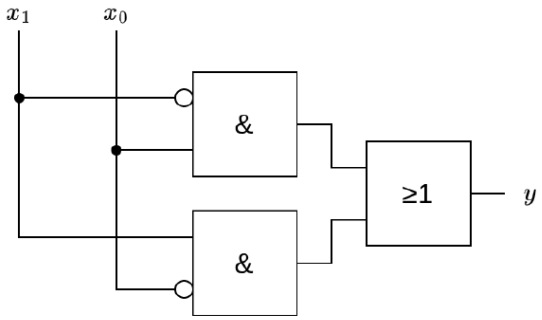
DNF-Schaltung: Beispiel XOR

Antivalenz (XOR)

x_1	x_0	$y = x_1 \nleftrightarrow x_0$
0	0	0
0	1	1
1	0	1
1	1	0

DNF: $y = (\overline{x_1} \wedge x_0) \vee (x_1 \wedge \overline{x_0})$

Zweistufige Schaltung gemäß DNF:



Konjunktive Normalform

03_combinational, p.13
Errinnerung: Disjunktion = ODER
- **Maxterm** ist ein UND aller Variablen und ist für genau eine Belegung 0.
- KNF = UND aller Maxterme
- !umgekehrt zum Minterm:
- wenn Eintrag = 0 $\rightarrow x_i$
- wenn Eintrag = 1 $\rightarrow \overline{x_i}$

Verbinden von DNF und KNF

- Jede boolesche Funktion hat genau eine DNF und eine KNF
- DNF und KNF beschreiben dasselbe Verhalten (der einen Wahrheitstafel), aber in unterschiedlicher Darstellung (mit unterschiedlichen booleschen Ausdrücken)
- Umformbar mit Hilfe von Gesetzen
- Beweis: 03_combinational, p.15

Zweistufiger Schaltungsentwurf

- Aus jeder Normalform lässt sich direkt eine Schaltung mit zwei Gatterstufen aufbauen.
- Für DNF und KNF ist eine direkte Umsetzung der Schaltung mit UND-/ODER- und NICHT-Gattern immer möglich → zweistufiger Schaltungsentwurf

Beispiel XOR

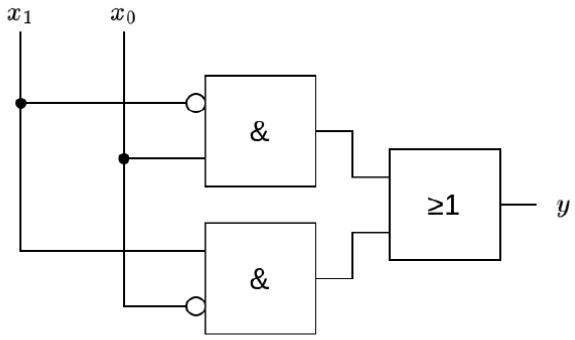
DNF

Antivalenz (XOR)

x_1	x_0	$y = x_1 \nleftrightarrow x_0$
0	0	0
0	1	1
1	0	1
1	1	0

DNF: $y = (\overline{x_1} \wedge x_0) \vee (x_1 \wedge \overline{x_0})$

Zweistufige Schaltung gemäß DNF:



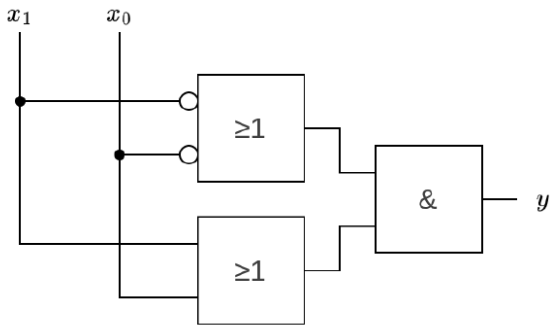
KNF

Antivalenz (XOR)

x_1	x_0	$y = x_1 \nleftrightarrow x_0$
0	0	0
0	1	1
1	0	1
1	1	0

KNF: $y = (x_1 \vee x_0) \wedge (\overline{x}_1 \vee \overline{x}_0)$

Zweistufige Schaltung gemäß KNF:



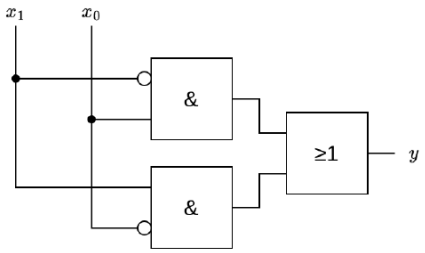
- Die Bubbles an den Eingängen sind NOT

Vereinfachen:

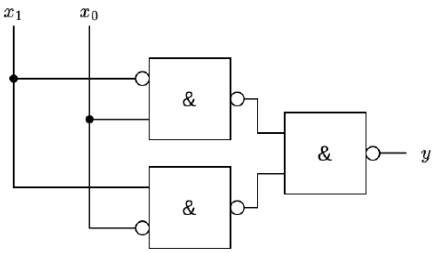
- NAND only Realisierung
- Benutzen von nur einem Gattertyp → Aufbau nur aus NAND
- NAND (und auch NOR) sind alleine bereits vollständige Operatorensysteme

Vorgehen:

1. Analytisch: Mit De Morgan umformen, bis nur NANDs übrig sind.
2. Grafisch: mittels Bubble Pushing
   - Du setzt auf jede Leitung zwischen Stufe 1 und 2 zwei Bubbles.
   - Zwei Negationen heben sich auf, also ändert sich nichts.
   - Die erste Bubble sitzt am Ausgang des UND und macht daraus ein NAND.
   - Die zweite sitzt am Eingang des ODER.
   - Ein ODER mit negierten Eingängen ist ein NAND, denn $\overline{A} \vee \overline{B} = \overline{A \wedge B}$



DNF für XOR



NAND-basierte Schaltung

Minimieren

- 2^n Minterme sind sehr viele
- Ziel: finden einer kürzeren gleichwertigen Darstellung derselben booleschen Funktion

Vereinfachen durch algebraisches Umformen Boolesche Ausdrücke lassen sich mit den bekannten Rechenregeln (De Morgan, Assoziativ-, Distributivgesetz, …) umformen und damit vereinfachen.

Beispiel:

$$
\begin{aligned}
y &= x_1 \vee \overline{\overline{x_1 \wedge x_0} \vee x_0} \\
&= x_1 \vee (x_1 \wedge x_0 \wedge \overline{x_0}) \\
&= x_1 \vee (x_1 \wedge (x_0 \wedge \overline{x_0})) \\
&= x_1 \vee (x_1 \wedge 0) \\
&= x_1
\end{aligned}
$$

$$
\begin{aligned}
y &= x_1 \vee \overline{\overline{x_1 \wedge x_0} \vee x_0} \\
&= x_1 \vee \overline{(\overline{x_1} \vee \overline{x_0}) \vee x_0} \\
&= x_1 \vee \overline{\overline{x_1} \vee 1} \\
&= x_1 \vee (x_1 \wedge \overline{1}) \\
&= x_1 \vee (x_1 \wedge 0) = x_1
\end{aligned}
$$

Grafisch überprüfen mithilfe ablesen an der Wahrheitstafel: 03_combinational, p.27

KV Diagramme

03_combinational, p.30
Ein grafisches Verfahren, um benachbarte Einsen systematisch zu größeren Blöcken zusammenzufassen.
- Benachbarte Einsen lassen sich in einem Term zusammenfassen
- Variablenbelegungen heißen benachbart, wenn sie sich in genau einer freien Variable unterscheiden

Einführungsübung: 03_combinational, p.29

Lösung:
[Übung Folie 29.png not found]

x_2	x_1	x_0	y	Minterme
0	0	0	1	$\overline{x}_2 \wedge \overline{x}_1 \wedge \overline{x}_0$
0	0	1	1	$\overline{x}_2 \wedge \overline{x}_1 \wedge x_0$
0	1	0	0	
0	1	1	0	
1	0	0	1	$x_2 \wedge \overline{x}_1 \wedge \overline{x}_0$
1	0	1	1	$x_2 \wedge \overline{x}_1 \wedge x_0$
1	1	0	1	$x_2 \wedge x_1 \wedge \overline{x}_0$
1	1	1	0	

Ausgangspunkt:

- Boolesche Funktion $y = f(x_2, x_1, x_0)$
- Definiert durch Wahrheitstafel
- Minterme bestimmen

Das **Ergebnis** ist: $y = \overline{x_1} \vee x_2\overline{x_0}$

→ Von Hand musst du alle Paare prüfen, und das wird schnell unübersichtlich. Im KV-Diagramm liegen Nachbarn nebeneinander, und du siehst diese Blöcke/ das Ergebnis direkt.

Deswegen KV Diagramme

- Reihenfolge 00, 01, 11, 10
- nicht die Binärreihenfolge 00, 01, 10, 11
- ermöglichen das Verbinden benachbarter Felder (Einsen) zu Blöcken, aus denen letztendlich eine minimierte Form abgelesen werden kann

Blöcke:

- Benachbarte Einsen werden zu Blöcken der Größe 2^k zusammengefasst
- Rechteckig
- Maximaler Block heißt Primblock mit dem Primimplikant als Term
- Je größer der Block
  - desto weniger Variablen im Ergebnis
  - kleiner die Schaltung

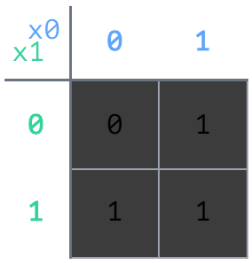
Beispiel:

- Die drei Einsen lassen sich zu zwei überlappenden 2er-Blöcken zusammenfassen
- Block 1: rechte Spalte Primimplikant x0
- Block 2: untere Zeile - Primimplikant x1:
- Resultierende minimierte Form $y = x_1 \vee x_0$

Wahrheitstafel für ODER

x_1	x_0	y
0	0	0
0	1	1
1	0	1
1	1	1

KV-Diagramm

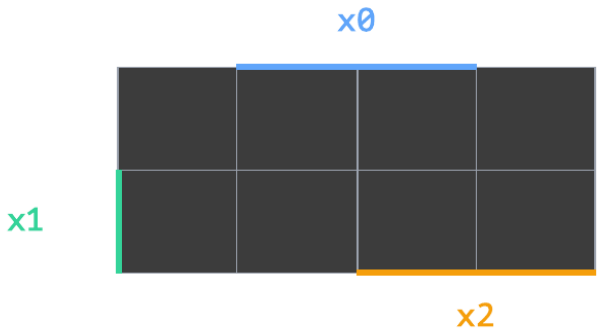


3 Variablen

- Spiegeln nach Rechts:

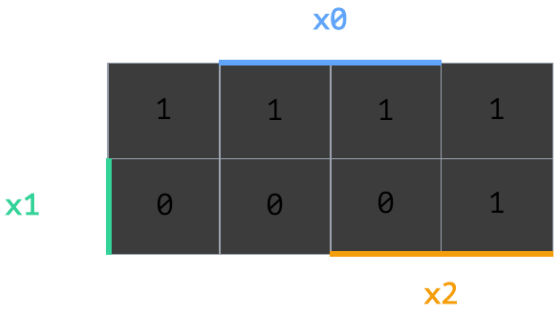
dec	x_2	x_1	x_0	y
0	0	0	0	1
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

🔧 Befüllen des KV-Diagramms



0	**1**	**5**	**4**
2	**3**	**7**	**6**

- Sind **alle** Einsen des KV-Diagramms mindestens einmal **überdeckt**, ergibt die ODER-Verknüpfung der Primimplikanten die minimierte DNF
- 💡 Die Überdeckung der Einsen ist **nicht eindeutig**, d.h. es existieren mehrere minimale Lösungen



- Minimierte DNF:

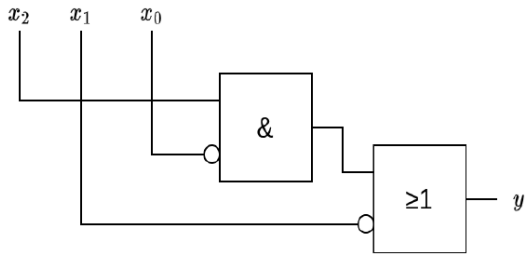
  $y = \overline{x_1} \vee (x_2 \wedge \overline{x_0})$

- Diese Form hat nur noch 2 Terme statt der ursprünglichen 5 Minterme

Schaltung zur minimierten Darstellung

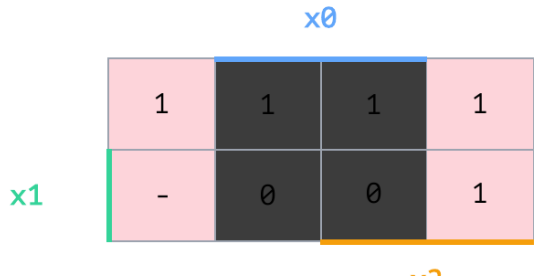
- Die minimierte DNF $y = \overline{x_1} \vee (x_2 \wedge \overline{x_0})$ lässt sich direkt als Schaltung umsetzen

Zweistufige Schaltung der minimierten DNF:



Don’t-Care-Belegungen

- Wenn Variablenbelegungen **nie** auftreten, ist der Funktionswert einer Schaltung dafür irrelevant
- Der Funktionswert ist **egal** ("don’t care") - er wird mit "-" in Wahrheitstafel und KV-Diagramm markiert
- Beim Bilden von Blöcken kann man Don’t-Cares nach Belieben als 0 oder 1 interpretieren, um möglichst **große** Primblöcke zu erzeugen



- Block 1: Primimplikant $\rightarrow \overline{x_1}$
- Block 2: Primimplikant $\rightarrow \overline{x_0}$ (mit Don’t Care und über Rand des Diagramms hinweg)
- Minimierte DNF: $y = \overline{x_1} \vee \overline{x_0} = \overline{x_1 \wedge x_0}$

Das Ergebnis ist y = ¬x1 ¬x0 = ¬(x1 x0), also **ein einziges NAND**. Ohne den Don’t-Care wäre es ¬x1 (x2 ¬x0) gewesen.